

FLOOD-AI

Intelligent Agent Layer — Architecture Reference

Physics-grounded flood simulation with a multi-agent AI observer layer. This document covers the complete architecture of 20 implemented files and 15 upcoming files, including data flow, design decisions, and inter-module dependencies.

C++ FLOOD-ENGINE · Python FLOOD-AI · Qt Flood3D Frontend

System Overview

FLOOD-AI is a standalone Python service that operates as an intelligent observation and decision layer on top of a physics-based C++ flood simulation engine. The three components of the full system are completely decoupled from each other by design.

The Three Layers

FLOOD-ENGINE (C++) — The physics core. Runs DEM-based shallow water equations, flow accumulation, infiltration, and rainfall-runoff modeling. Outputs a 2D grid of flood depth, flow velocity, and rainfall intensity at each timestep. Compiled as a DLL. This layer never changes due to AI integration.

FLOOD-AI (Python) — The intelligence layer. Connects to the engine output via socket, parses grid data, runs regional AI agents, coordinates global risk decisions, fires alerts, and generates reports. Completely separate process.

Flood3D (Qt/C++) — The visualization frontend. Renders 3D terrain, water layers, risk markers, and road networks. Connects to FLOOD-AI via REST API and WebSocket to display live agent decisions and alerts.

Core Design Principle

Agents do not simulate physics. They read physics outputs and interpret them. This is the fundamental architectural separation that makes the system reliable — the simulation is always deterministic and grounded in real hydrology. The AI layer adds contextual interpretation, memory, pattern detection, and natural language reporting on top of that ground truth.

Data Flow

1. Engine runs one timestep and sends a JSON tick over socket
2. `dll_reader.py` receives the raw packet and deserializes it
3. `timestep_listener.py` fires the agent cycle
4. `agent_runner.py` runs all `RegionAgents` in parallel
5. Each `RegionAgent` slices its grid, computes stats, scores risk, detects anomalies
6. All region reports collected by `CoordinatorAgent`
7. `Coordinator` propagates risk downstream using terrain graph, generates alerts
8. `AlertAgent` dispatches alerts with cooldown management
9. `SnapshotStore` saves the full timestep result to disk
10. `WebSocket` pushes tick update to Qt frontend in real time

Implemented Files — 20 Files

config.py

Role: Single source of truth for all environment configuration.

Uses pydantic-settings BaseSettings to load all values from the .env file with automatic type validation. Exposes a singleton settings object via an LRU-cached get_settings() function. Every other module imports settings from here — never reads os.environ directly. This ensures that if a required variable is missing, the system fails fast at startup with a clear validation error rather than at runtime. Contains LLM API credentials, engine host/port, API server config, file paths, agent tuning parameters like memory window size and max regions.

```
Key fields: llm_api_key, flood_engine_host, flood_engine_port, agent_memory_window,
max_regions
```

engine_bridge/engine_schema.py

Role: Defines the shape of all data coming out of the C++ engine.

Pydantic models that exactly mirror what the engine sends over the socket. The base GridSnapshot contains rows, cols, cell_size_m, and the 2D data list. FloodGridSnapshot extends this with max_depth_m and flooded_cell_count. FlowGridSnapshot adds max_velocity_ms. RainfallGridSnapshot adds total_rainfall_mm. EngineTickPayload is the top-level container that wraps all three grid types plus timestep and elapsed_seconds. Every downstream module works with these typed models — no raw dicts touching agent logic.

```
Key models: GridSnapshot, FloodGridSnapshot, FlowGridSnapshot, RainfallGridSnapshot,
EngineTickPayload
```

engine_bridge/grid_parser.py

Role: Converts typed engine models into numpy arrays and computes region statistics.

The bridge between Pydantic models and numerical computation. parse_flood_grid(), parse_flow_grid(), parse_rainfall_grid() each convert a snapshot's 2D list into a numpy float64 array. extract_region_slice() takes a grid and a bounding box to extract just the cells belonging to one geographic region — this is how each RegionAgent gets its local view of the world. compute_region_stats() returns mean, max, min, flooded cell count, mean of nonzero cells, and total cells — the statistical summary that agents use for risk scoring. grid_to_flat() serializes a grid back for storage.

engine_bridge/dll_reader.py

Role: Socket connection to the C++ engine. Reads newline-delimited JSON tick messages.

Maintains a TCP socket connection to the engine at the configured host and port. Runs a background daemon thread that continuously reads from the socket into a string buffer. When a newline is detected, it splits off a complete message, parses it as JSON, validates it into an EngineTickPayload, and fires the registered on_tick callback. Handles partial reads gracefully via buffer accumulation. If the connection

drops, logs the error and stops the loop cleanly. The `on_tick()` method lets the `timestep_listener` register its handler without `dll_reader` knowing anything about agents. A singleton `dll_reader` instance is exported.

engine_bridge/timestep_listener.py

Role: Wires the engine tick event to the agent cycle. The glue between bridge and agents.

Contains a single `_on_engine_tick()` callback that checks session state, advances the session timestep counter, then calls `agent_runner.run_cycle()` with the payload. The `wire()` function registers this callback on `dll_reader`. This is the only place where the engine bridge and the agent layer touch — keeping both sides ignorant of each other's internals. If the session is not active, ticks are silently ignored.

core/schemas.py

Role: Shared data models used across all modules. The lingua franca of the system.

Defines all domain entities as Pydantic models with UUID auto-generation and UTC timestamps. RiskLevel is a five-value enum: NONE, LOW, MEDIUM, HIGH, CRITICAL. AlertSeverity has four levels: INFO, WARNING, CRITICAL, EMERGENCY. BoundingBox defines a rectangular grid region by row/column indices. RegionConfig holds a region's identity, bounding box, upstream and downstream IDs for graph traversal, and optional physical metadata. RegionObservation is what a RegionAgent produces per timestep — stats plus detected risk and anomalies. AlertEvent is a dispatched alert with acknowledgement state. SimulationRun tracks a run's lifecycle. CoordinatorDecision is the output of the coordinator per timestep.

core/session.py

Role: Tracks the lifecycle of one simulation run — active state, timestep counter, run metadata.

SimulationSession wraps a SimulationRun and exposes start(), stop(), tick(), is_active(), current_timestep(), and get_run(). start() creates a new SimulationRun with a fresh UUID. Each tick() call from the timestep listener increments the counter. stop() records the end time. A singleton session instance is exported. The API endpoints use this to gate operations — you cannot stop a session that isn't running, and you cannot start one that already is.

core/region_registry.py

Role: The master list of all geographic regions being monitored.

RegionRegistry stores RegionConfig objects keyed by region_id. Provides register(), get(), all(), ids(), count(), remove(), clear(). Also provides get_downstream() and get_upstream() which resolve region IDs from a region's relationship lists — these are used by the CoordinatorAgent for risk propagation. A singleton registry is exported. Regions are registered at startup from DEM/boundary data before any simulation starts.

agents/base_agent.py

Role: Abstract base class enforcing the observe-analyze-decide contract on all agents.

BaseFloodAgent is an ABC with three abstract methods: observe() takes an EngineTickPayload and returns a RegionObservation, analyze() takes that observation and returns any intermediate analysis structure, decide() takes the analysis and returns the final decision dict. The concrete run() method chains all three. Also manages active/inactive state per agent. Every agent in the system inherits from this — enforcing architectural consistency regardless of how each agent implements its logic.

agents/agent_memory.py

Role: Gives each agent short-term working memory and long-term event memory.

Short-term memory is a deque with maxlen set to agent_memory_window (default 20 timesteps). Each push() adds a RegionObservation and updates the peak risk seen. If the risk is HIGH or CRITICAL, it commits a summary to long-term memory as a JSON-serializable dict. flood_trend() looks at the last 5 short-term observations and computes the average change in mean flood depth per timestep — returning

'rising', 'falling', or 'stable'. `recent(n)` returns the last `n` observations. `save()` persists long-term events to disk under the run folder.

agents/region_agent.py

Role: The primary observer agent. One instance per geographic zone.

RegionAgent is the most important class in the system. `observe()` uses `grid_parser` to extract and compute stats for its specific bounding box from the three grids. `_score_risk()` maps max flood depth to a RiskLevel using calibrated thresholds: 0.1m = LOW, 0.3m = MEDIUM, 0.6m = HIGH, 1.2m = CRITICAL. `_detect_anomalies()` compares current stats to the last observation in memory to detect rapid rise events (threshold 0.08m/step), checks if flooded cell coverage exceeds 60%, and flags extreme depths above 2m. `analyze()` pushes the observation to memory and computes trend and peak. `decide()` assembles the final report dict consumed by CoordinatorAgent.

```
Thresholds — LOW: 0.1m | MEDIUM: 0.3m | HIGH: 0.6m | CRITICAL: 1.2m | Rapid rise: 0.08m/step
```

agents/coordinator_agent.py

Role: Global decision maker. Aggregates region reports and propagates risk downstream.

`process()` receives all region reports for a timestep. It first maps each report to a RiskLevel. Then `_propagate_risks()` checks which regions are HIGH or CRITICAL and looks up their downstream neighbors via the registry — if a downstream region is currently below MEDIUM, it is elevated to MEDIUM and a propagation warning is recorded. `_compute_global_risk()` takes the maximum risk across all regions. `_generate_alerts()` creates AlertEvents for CRITICAL regions (EMERGENCY severity), HIGH regions (CRITICAL severity), and any region with a rapid rise trend (WARNING severity). The full CoordinatorDecision is returned and stored as `last_decision` for the WebSocket to read.

agents/alert_agent.py

Role: Alert dispatcher with per-region cooldown logic.

`dispatch()` iterates the alert list and for each alert checks if the region is within its cooldown window. Cooldown varies by severity: EMERGENCY = 0 seconds (always fires), CRITICAL = 10s, WARNING = 30s, INFO = 60s. If not cooling down, the alert is appended to history, the `last_fired` time is updated, and all registered channel callbacks are invoked. Channels are pluggable — the WebSocket pusher registers itself as a channel. `active_alerts()` returns unacknowledged alerts. `acknowledge()` flips the acknowledged flag by `alert_id`.

agents/agent_runner.py

Role: Orchestrator. Runs all region agents per tick and feeds results to the coordinator.

`build_agents()` is called once at session start — it reads all regions from the registry and instantiates one RegionAgent per region. `run_cycle()` is called by `timestep_listener` on every tick. It iterates all active agents, calls `agent.run(payload)`, catches and logs individual agent failures without crashing the whole cycle, collects all reports, sends them to `coordinator.process()`, dispatches alerts via `alert_agent`, and saves the snapshot via `snapshot_store`. `get_agent()` allows the API to inspect individual agents.

agents/report_agent.py

Role: Post-simulation analyst. Generates structured JSON summary and LLM-written narrative report.

`generate()` reads all snapshots from `snapshot_store`, builds a summary dict containing total timesteps, total alerts, overall peak risk, list of critical regions, and per-region peak risk and max flood depth. It then calls the LLM via `llm_client` with a structured prompt asking for a professional flood risk assessment. The result is stored in both JSON and markdown format and saved to the run folder. `query()` allows interactive questions — it serializes the report JSON as context and asks the LLM to answer strictly based on that data, preventing hallucination.

services/snapshot_store.py

Role: Stores and retrieves per-timestep simulation snapshots in memory and on disk.

`save_snapshot()` builds a lightweight snapshot dict from the timestep number, coordinator decision, and region reports — keeping only the fields relevant for analysis rather than the full grid data. Snapshots are kept in memory as a list for fast access by `ReportAgent` and also written to disk as JSON files named by zero-padded timestep (`t000001.json`, `t000002.json` etc.) under the run's snapshots folder. `latest()` returns the most recent snapshot for the `WebSocket` to broadcast. `all()` gives `ReportAgent` the full history.

services/llm_client.py

Role: Thin wrapper around the OpenAI-compatible LLM API.

`ask()` constructs a messages array with an optional system prompt and user prompt, calls the configured model, and returns the response text. Temperature is fixed at 0.3 to keep outputs deterministic and factual. `ask_json()` wraps `ask()` and strips markdown code fences before JSON parsing — defensive handling for models that wrap JSON in backtick blocks. Errors are logged and `None` is returned rather than throwing, letting callers decide how to handle LLM unavailability gracefully.

api/websocket.py

Role: Real-time push layer to the Qt frontend via WebSocket.

`ConnectionManager` maintains a list of connected `WebSocket` clients. `broadcast()` sends a JSON message to all connected clients and removes any that have disconnected. `push_tick_update()` is called after each agent cycle — it reads the coordinator's last decision and broadcasts a tick event containing global risk, all region risks, and any alerts issued that timestep. `push_alert()` can broadcast individual alert events. Dead connections are pruned automatically. The Qt frontend connects once and receives a continuous stream of JSON events.

api/endpoints/session.py

Role: Session lifecycle REST endpoints consumed by Qt.

`POST /session/start` initializes the session, builds all region agents, sets the snapshot store run ID, connects to the engine, wires the timestep listener, and starts the socket listener thread. Returns the `run_id` and engine connection status. `POST /session/stop` gracefully stops the session and socket connection, returns total timesteps processed. `GET /session/status` returns the full current state including active flag, timestep, region count, agent count, and active alert count.

api/endpoints/agents.py

Role: Agent inspection endpoints for the Qt frontend to query agent state.

`GET /agents/` lists all registered regions with their agent active state. `GET /agents/{region_id}` returns the full agent state including peak risk, current flood trend, all long-term memory events, and the last 5 observations. `GET /agents/{region_id}/risk` returns just the current risk assessment — the lightweight endpoint Qt polls for map overlay coloring.

Future Files — 15 Remaining

The following 15 files complete the system. They are designed and their contracts are defined — implementation comes next in sequence.

API Layer — 3 files

`api/endpoints/alerts.py`

Three endpoints: GET `/alerts/active` returns all unacknowledged alert events, GET `/alerts/history` returns the full alert log for the current run, POST `/alerts/ack/{alert_id}` marks an alert as acknowledged. The Qt map view uses `active_alerts` to show the alert badge count and alert panel. Acknowledging from the UI calls this endpoint which updates the `alert_agent` state.

`api/endpoints/report.py`

Three endpoints: GET `/report/summary` returns the structured JSON report from `report_agent`, GET `/report/markdown` returns the full narrative text, POST `/report/query` accepts a JSON body with a question field and returns `report_agent.query(question)`. This allows the Qt interface to have an interactive chat panel where the engineer can ask questions about the simulation results.

`api/router.py`

The FastAPI application root. Creates the FastAPI app, includes all endpoint routers with their prefixes, mounts the WebSocket endpoint at `/ws`, and adds CORS middleware for Qt's local HTTP client. This is the single file that assembles the complete API surface.

Core Layer — 2 files

`core/data_loader.py`

Called once at startup before session start. Reads DEM files from `data/dem/`, watershed boundary GeoJSON from `data/boundaries/`, road network from `data/roads/`, and building footprints from `data/buildings/`. Uses this data to automatically cluster grid cells into geographic regions via `services/region_clusterer.py` and register them in the `RegionRegistry`. Also populates physical metadata on each `RegionConfig` — elevation mean, area. This is what makes the system data-driven rather than requiring manual region definition.

`core/run_manager.py`

Manages run artifact folder creation and cleanup. On session start, creates `output/runs/{run_id}/` with subdirectories `snapshots/`, `memory/`, `report/`, `alerts/`, `logs/`. Writes the `run_manifest.json` with start time, region count, and config snapshot. On session stop, finalizes the manifest with end time and total timesteps. Provides `list_runs()` and `get_run_manifest()` for a future run history API.

Services Layer — 4 files

services/risk_analyzer.py

A physics-grounded risk scoring service that goes beyond simple depth thresholds. Combines flood depth, flow velocity, terrain slope, and rainfall intensity into a composite risk score. Depth alone is insufficient — 0.5m of fast-moving water is more dangerous than 0.8m of standing water. The formula weights velocity heavily for road and infrastructure risk. Used by RegionAgent as a secondary scoring method alongside the threshold-based `_score_risk()` for cross-validation.

services/alert_router.py

Formalizes the alert channel registration system. Provides a `register_channel()` function and built-in channel implementations: `websocket_channel()` pushes to the WebSocket manager, `log_channel()` writes structured log entries, `file_channel()` appends to an alert log file per run. AlertAgent calls `alert_router.route(alert)` instead of managing channels directly. This separates routing logic from alert dispatching logic.

services/region_clusterer.py

Automatically divides the simulation grid into geographic regions using watershed boundary data. Takes the DEM grid and boundary GeoJSON, assigns each grid cell to its watershed region, computes bounding boxes, and returns a list of RegionConfig objects. This removes the need to manually define regions — the real watershed topology drives the agent territory boundaries. Upstream/downstream relationships are derived from flow direction analysis on the DEM.

services/snapshot_store.py — already implemented

Graph Layer — 4 files

The graph layer is the most architecturally significant remaining component. It gives the system a spatial knowledge structure — not just a flat list of regions but a connected graph encoding how water moves between them. CoordinatorAgent currently uses simple registry upstream/downstream lists. After the graph layer is built, it will use `graph_query` to traverse multi-hop propagation paths.

graph/graph_schema.py

Pydantic models for the terrain graph. TerrainNode holds `region_id`, `name`, centroid coordinates, elevation mean, and area. FlowEdge connects two nodes with a `flow_weight` representing the estimated water transfer capacity. WatershedCluster groups nodes that share a top-level catchment. These models are the serializable representation of the graph stored to disk.

graph/terrain_graph_builder.py

Reads RegionConfig objects from the registry and DEM flow direction data to construct a NetworkX directed graph. Each region becomes a node. An edge is drawn from region A to region B if water flows from A's cells into B's cells according to the flow direction grid. Edge weights are proportional to the number of flow-connected cells. The graph is built once at startup and saved via `graph_storage`.

graph/graph_storage.py

Serializes and deserializes the NetworkX graph to JSON using NetworkX's node-link format. `save_graph()` writes to `graph/graph.json` under the run folder. `load_graph()` reconstructs the graph on startup if a cached

version exists, avoiding rebuilding from scratch each run. Also writes a `graph_summary.json` with node count, edge count, and longest flow path.

graph/graph_query.py

Query facade over the NetworkX graph. `get_upstream_regions(region_id, hops)` returns all regions within N hops upstream. `get_downstream_regions(region_id, hops)` does the same downstream. `get_flow_path(source, target)` returns the ordered list of regions water would traverse between two points. `get_at_risk_downstream(region_id)` returns all downstream regions that would be affected if the given region floods. These methods replace the simple list lookups in `CoordinatorAgent` with proper graph traversal.

main.py — System Entry Point

The final file. Initializes logging via loguru, loads settings, calls `data_loader` to read terrain data and populate the registry, builds the terrain graph, creates the FastAPI app via `api/router.py`, registers the WebSocket alert channel on `alert_agent`, and starts the uvicorn server. This is the single command that brings the entire FLOOD-AI layer online: `python main.py`

```
python main.py
```

That single command boots the full system — data loaded, graph built, agents ready, API live, WebSocket open.

Module Dependency Map

Understanding which modules depend on which is critical for implementing files in the correct order and for debugging import errors.

Foundation (no dependencies)

`config.py` → imported by everything
`core/schemas.py` → imported by agents, api, services
`engine_bridge/engine_schema.py` → imported by bridge and agents

Engine Bridge

`grid_parser.py` → needs engine_schema
`dll_reader.py` → needs engine_schema, config
`timestep_listener.py` → needs dll_reader, agent_runner, session

Core

`session.py` → needs schemas
`region_registry.py` → needs schemas
`data_loader.py` → needs region_registry, region_clusterer
`run_manager.py` → needs config, session

Agents

`base_agent.py` → needs schemas, engine_schema
`agent_memory.py` → needs schemas, config
`region_agent.py` → needs base_agent, agent_memory, grid_parser, schemas
`coordinator_agent.py` → needs schemas, region_registry
`alert_agent.py` → needs schemas
`report_agent.py` → needs snapshot_store, llm_client, schemas
`agent_runner.py` → needs region_agent, coordinator, alert_agent, snapshot_store

Services

`snapshot_store.py` → needs schemas, config
`llm_client.py` → needs config
`risk_analyzer.py` → needs schemas
`alert_router.py` → needs schemas, websocket
`region_clusterer.py` → needs schemas, numpy

Graph

`graph_schema.py` → needs schemas

Recommended Implementation Order

Follow this sequence for the remaining 15 files. Each file only imports things already written.

Step 1 — `graph/graph_schema.py`

No dependencies. Defines graph data models needed by all graph files.

Step 2 — `graph/terrain_graph_builder.py`

Needs `graph_schema` and `region_registry`, both done.

Step 3 — `graph/graph_storage.py`

Needs `graph_schema` and `config`, both done.

Step 4 — `graph/graph_query.py`

Needs `graph_schema` and `networkx`. Complete the graph layer.

Step 5 — `services/risk_analyzer.py`

Needs only schemas. Standalone scoring logic.

Step 6 — `services/region_clusterer.py`

Needs schemas and `numpy`. Used by `data_loader`.

Step 7 — `core/data_loader.py`

Needs `region_registry` and `region_clusterer`, both now done.

Step 8 — `core/run_manager.py`

Needs `config` and `session`, both done.

Step 9 — `services/alert_router.py`

Needs schemas and `websocket`.

Step 10 — `api/endpoints/alerts.py`

Needs `alert_agent`, already done.

Step 11 — `api/endpoints/report.py`

Needs `report_agent`, already done.

Step 12 — `api/router.py`

Needs all endpoints. Assembles the full API.

Step 13 — `main.py`

Needs everything. Written last.

After `main.py` is written, the system is fully operational. Connect FLOOD-ENGINE to port 5050, run `python main.py`, and call `POST /session/start` from Qt to begin a live simulation with full AI agent observation.